

Lecture 23 - Dimensionality Reduction with Principal Component Analysis (PCA) & Introduction to Artificial Neural Networks (ANNs)

Principal Component Analysis (PCA)

A very common approach (and one of the simplest approaches) to **dimensionality reduction** is **Principal Component Analysis** (or **PCA**).

- PCA takes data from *sensor coordinates* to *data centric coordinates* using linear transformations.

PCA uses a **linear transformation** to **minimize the redundancy** of the resulting transformed data (by ending up with data that is uncorrelated).

- This means that every transformed dimension is more informative.
- In this approach, the dimensionality of the space is still the same as the original data, but the space of features are now arranged such that they contain the most information.

If we wish to reduce dimensionality of our feature space, we can choose only the features that carry over the most information in the linearly transformed space.

- In other words, PCA will find the underlying **linear manifold** that the data is embedded in.

PCA finds the directions of maximum variance in high-dimensional data and projects it onto a new subspace with equal or fewer dimensions than the original one.

There are a couple of points-of-view on how to find A :

1. Maximum Variance Formulation
2. Minimum-error Formulation

1. PCA as Maximum Variance Formulation

Consider the data X comprised on N data samples in a D -dimensional space, so X is a matrix of size $D \times N$.

The **first step** in PCA is to centralize or demean X . This will be to guarantee that all features will have the same impact and not weight more only because their range of values is much larger (example, age vs income).

- Without loss of generality, let's assume that we subtracted the mean to the input data, X . Now, X has zero mean.

The **second step** is to find the linear transformation A that transforms X to a space where features are:

1. Uncorrelated (preserve all dimensions)
2. reduced (dimensionality reduction)

$$Y = AX$$

where A is a $D \times D$ matrix, X is a $D \times N$ data matrix and therefore Y is also a $D \times N$ transformed data matrix.

The variance of the transformed data Y is given by:

$$\begin{aligned} R_y &= E[YY^T] \\ &= E[AX(AX)^T] \\ &= E[AXX^T A^T] \\ &= AE[XX^T]A^T \\ &= AR_x A^T \end{aligned}$$

Note that we are computing the variance along the dimensions of Y , therefore, R_y is a $D \times D$ matrix. Similarly, R_X represents the covariance of the data X . Covariance matrices are symmetric therefore $R_X = R_X^T$ and $R_Y = R_Y^T$.

Similarly, R_X represents the covariance of the data X .

If we write A in terms of vector elements:

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} = \begin{bmatrix} \vec{a}_1^T \\ \vec{a}_2^T \end{bmatrix}$$

Then,

$$\begin{aligned} R_y &= \begin{bmatrix} a_1^T \\ a_2^T \end{bmatrix} R_X \begin{bmatrix} a_1 & a_2 \end{bmatrix} \\ &= \begin{bmatrix} a_1^T R_X a_1 & a_1^T R_X a_2 \\ a_2^T R_X a_1 & a_2^T R_X a_2 \end{bmatrix} \end{aligned}$$

- If we want to represent the data in a space in which the features are **uncorrelated**, what shape does the covariance matrix have to take?

Diagonal! Why?

- Can we use the eigenvectors of R_X as our linear transformation A ?

Consider the case where we are trying to project the data X into a 1-dimensional space, so we are trying to find the direction a_1 where maximal data variance is preserved:

$$\arg_{a_1} \max a_1^T R_X a_1$$

We want this solution to be bounded (considering $a_1 = \infty$ would maximize), so we need to constraint the vector to have norm 1

$$\|a_1\|_2^2 = 1 \iff a_1^T a_1 = 1 \iff a_1^T a_1 - 1 = 0$$

Then, we using Lagrange Optimization:

$$\mathcal{L}(a_1, \lambda_1) = a_1^T R_X a_1 - \lambda_1 (a_1^T a_1 - 1)$$

Solving for a_1 :

$$\frac{\partial \mathcal{L}}{\partial a_1} = 0 \iff 2R_X a_1 - 2\lambda_1 a_1 = 0 \iff R_X a_1 = \lambda_1 a_1$$

- Does this look familiar?

This is stating that a_1 must be an eigenvector of R_X !

So coming back to the question "Can we use the eigenvectors of X as our linear transformation A ?" YES!

- If we left multiply by a_1^T and make use of $a_1^T a_1 = 1$:

$$a_1^T R_X a_1 = \lambda_1$$

So the variance will be maximum when we set the project direction a_1 equal to the eigenvector having the largest eigenvalue λ_1 .

- This eigenvector is known as the first **principal component**.
- As you may anticipate, the linear transformation A will be a matrix whose row entries are **sorted** the eigenvectors (sorted by their correspondent eigenvalue).

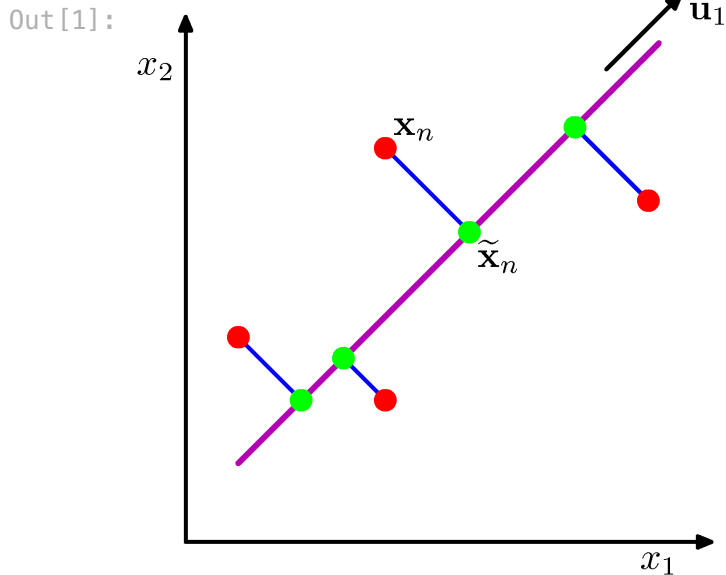
$$A = \begin{bmatrix} \vec{a}_1^T \\ \vec{a}_2^T \end{bmatrix}$$

where a_1 and a_2 are eigenvectors of R_X with correspondent eigenvalues λ_1 and λ_2 with $\lambda_1 > \lambda_2$.

2. PCA as Minimum-error Formulation

We can also look at PCA as a minimization of mean squared error.

```
In [1]: from IPython.display import Image
Image('figures/Figure12.2.png',width=300)
# Source: Bishop textbook
```



Consider X and a D -dimensional orthogonal basis $\mathbf{u}$:

$$\hat{x} = \sum_{i=1}^m y_i a_i$$

where $m < D$.

$$y_j = x^T a_j$$

where $A^T A = I$.

We want to minimize the residual error:

$$\epsilon = x - \hat{x} = \sum_{i=m+1}^D y_i a_i$$

- The objective function we will use is the mean square residual:

$$\begin{aligned}
J &= E \left[\|\epsilon\|_2^2 \right] \\
&= E \left[\left(\sum_{i=m+1}^D y_i a_i \right) \left(\sum_{i=m+1}^D y_i a_i \right) \right] \\
&= \sum_{j=m+1}^D E[y_j^2], \text{ because } a_i^T a_j = 0, \forall i \neq j \text{ and } a_i^T a_j = 1, \forall i = j \\
&= \sum_{j=m+1}^D E[(a_j^T \mathbf{x})(\mathbf{x}^T a_j)] \\
&= \sum_{j=m+1}^D a_j^T E[\mathbf{x}\mathbf{x}^T] a_j \\
&= \sum_{j=m+1}^D a_j^T R_x a_j
\end{aligned}$$

Minimize the error and incorporate Lagrange parameters for $A^T A = I$:

$$\begin{aligned}
\frac{\partial J}{\partial a_j} &= 2(R_x a_j - \lambda_j a_j) = 0 \\
R_x a_j &= \lambda_j a_j
\end{aligned}$$

So, the sum of the error is the sum of the eigenvalues of the unused eigenvectors. So, we want to select the eigenvectors with the m largest eigenvalues.

Pseudo-code for Implementing PCA

Consider the data X with N data points defined in a D -dimensional space, that is, X is a $D \times N$ matrix.

(Subtract the mean, $\mu = \frac{1}{N} \sum_{i=1}^N x_i$.)

1. Compute the covariance matrix R_X (by definition, the covariance already subtracts the data's mean). This matrix is of size $D \times D$.
2. Compute eigenvectors and eigenvalues of the matrix R_X , and store the sorted eigenvectors (e_i) in decreasing eigenvalue (λ_i) order.
3. Build the modal matrix $\mathbf{U} = [\mathbf{e}_1 \mid \mathbf{e}_2 \mid \dots \mid \mathbf{e}_D]$, where all the (unit-length) eigenvectors are stacked in columns, sorted by their respective eigenvalues, i.e., $\lambda_1 > \lambda_2 > \dots > \lambda_D$.

- For **uncorrelating the data**, preserve all D eigenvectors. Hence $\mathbf{U}$ is a $D \times D$ matrix.

- For **dimensionality reduction**, keep the top M eigenvectors with the largest eigenvalues. Hence $\mathbf{U}$ is a $D \times M$ matrix.
4. Apply the linear transformation: $\mathbf{y} = \mathbf{U}^T \mathbf{X}$. Here $\mathbf{y}$ is a matrix of size $M \times N$, where $M \leq D$.

Note that the formal definition of covariance already accounts for demeaning the data.

```
In [1]: import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
plt.style.use('bmh')
import scipy.stats as stats
import pandas as pd
```

```
In [2]: # Helper functions

def plotvec(*argv):
    colors=['k','b','r','g','c','m']
    xmin=0
    xmax=-1000000
    ymin=0
    ymax=-1000000
    origin=[0,0]
    # plt.figure()
    for e in enumerate(argv):
        i=e[0]
        arg=e[1]
        plt.quiver(*origin,*arg,angles='xy',scale_units='xy',scale=1,
                    color=colors[i%len(colors)])
        xmin=min(xmin,arg[0])
        xmax=max(xmax,arg[0])
        ymin=min(ymin,arg[1])
        ymax=max(ymax,arg[1])
    # plt.xlim(min(-1, xmin-1), max(1, xmax+1))
    # plt.ylim(min(-1, ymin-1), max(1, ymax+1))

def plot_contours(K,X=None, R=None):
    '''This function plots the contours of a Bivariate Gaussian RV with
    mean [0,0] and covariance K'''

    x = np.linspace(-4, 4, 100)
    y = np.linspace(-4, 4, 100)
    xm, ym = np.meshgrid(x, np.flip(y))
    if X is None:
        X = np.dstack([xm,ym])
    if R is not None:
        X = X@R

    G = stats.multivariate_normal.pdf(X,mean=[0,0],cov=K)

    plt.figure(figsize=(6,6))
    plt.contour(xm,ym,G, extent=[-3,3,-3,3],cmap='viridis');
```

```
def makerot(theta):
    '''This function creates a 2x2 rotation
    matrix for a given angle (theta) in degrees'''

    theta=np.radians(theta)

    R = np.array([[np.cos(theta), -np.sin(theta)],
                  [np.sin(theta), np.cos(theta)]])

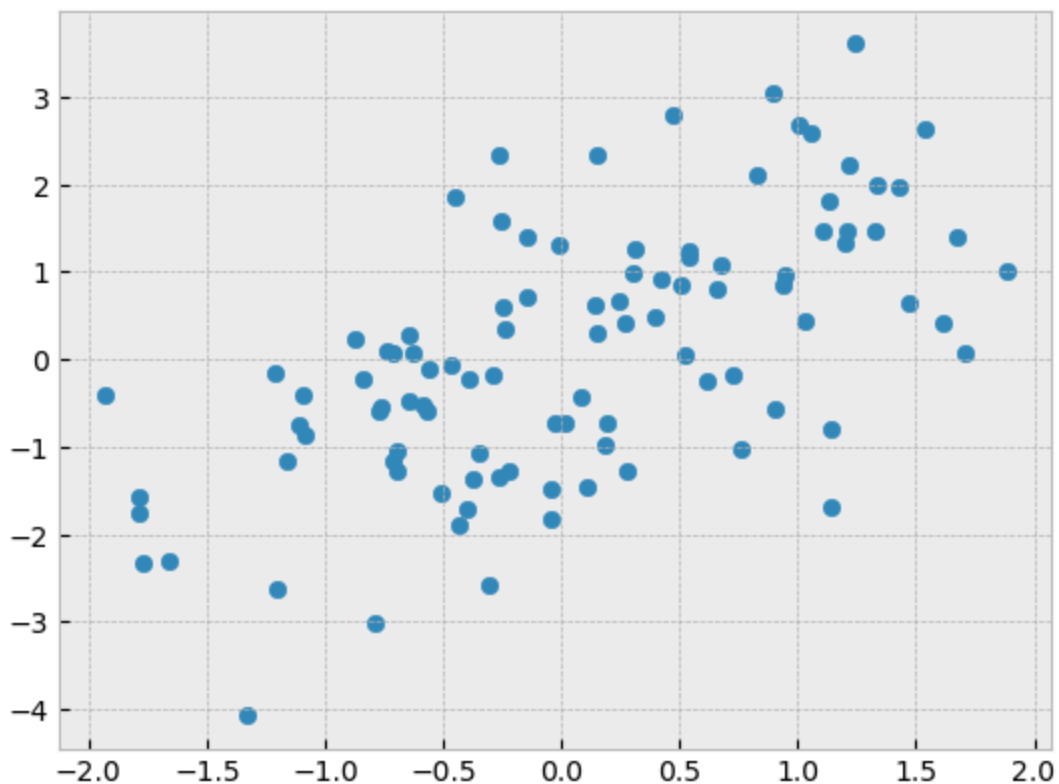
    return R
```

```
In [3]: data = stats.multivariate_normal([0,0],[[1,0.8],[0.8,2]]).rvs(size=100)

data.shape
```

```
Out[3]: (100, 2)
```

```
In [4]: plt.scatter(data[:,0],data[:,1]);
```

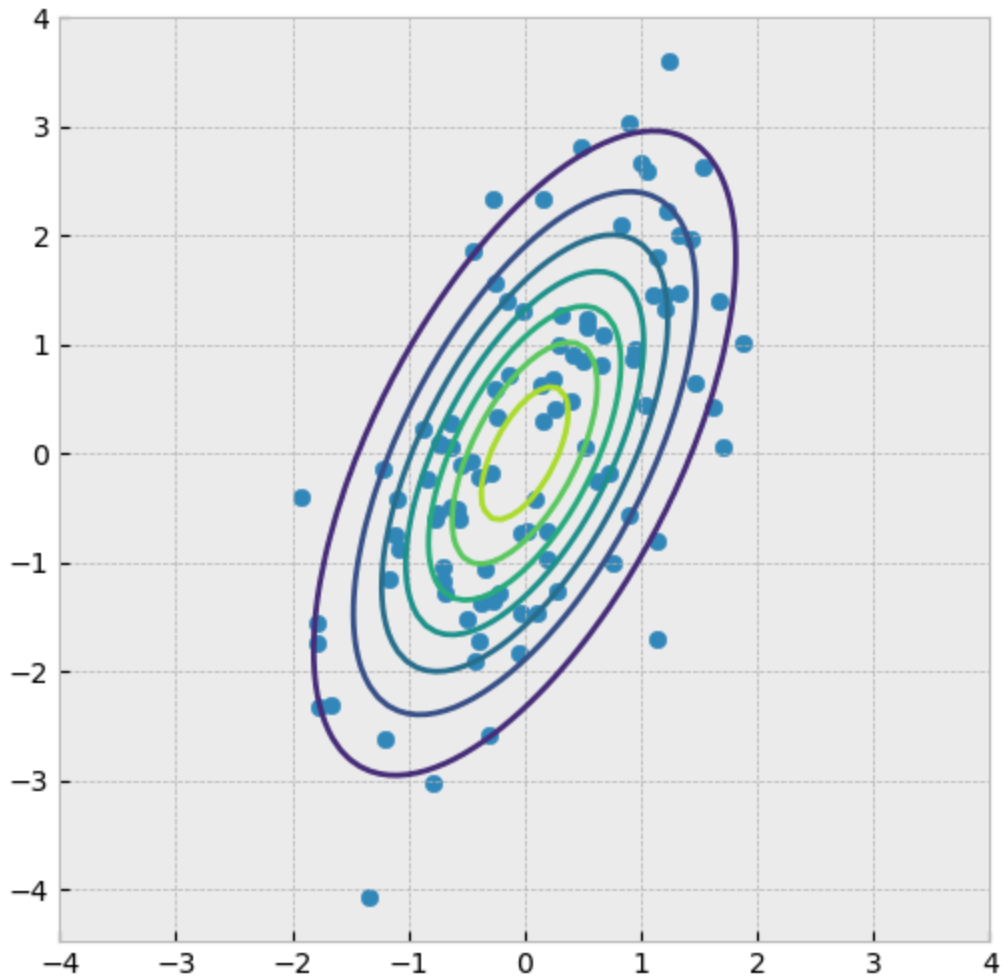


```
In [5]: K = np.cov(data.T)

K
```

```
Out[5]: array([[0.81031093, 0.80993393],
               [0.80993393, 2.15042973]])
```

```
In [6]: plot_contours(K)
plt.scatter(data[:,0],data[:,1]);
```



```
In [7]: # Eigendecomposition for Hermitian matrices (real, symmetric matrices -- cov
L, V = np.linalg.eigh(K)
```

```
L, V
```

```
Out[7]: (array([0.42919357, 2.53154708]),
        array([[ -0.90483076,  0.4257714 ],
               [ 0.4257714 ,  0.90483076]]))
```

```
In [8]: # Store eigenvalues and eigenvectors in decreasing order of the eigenvalues
```

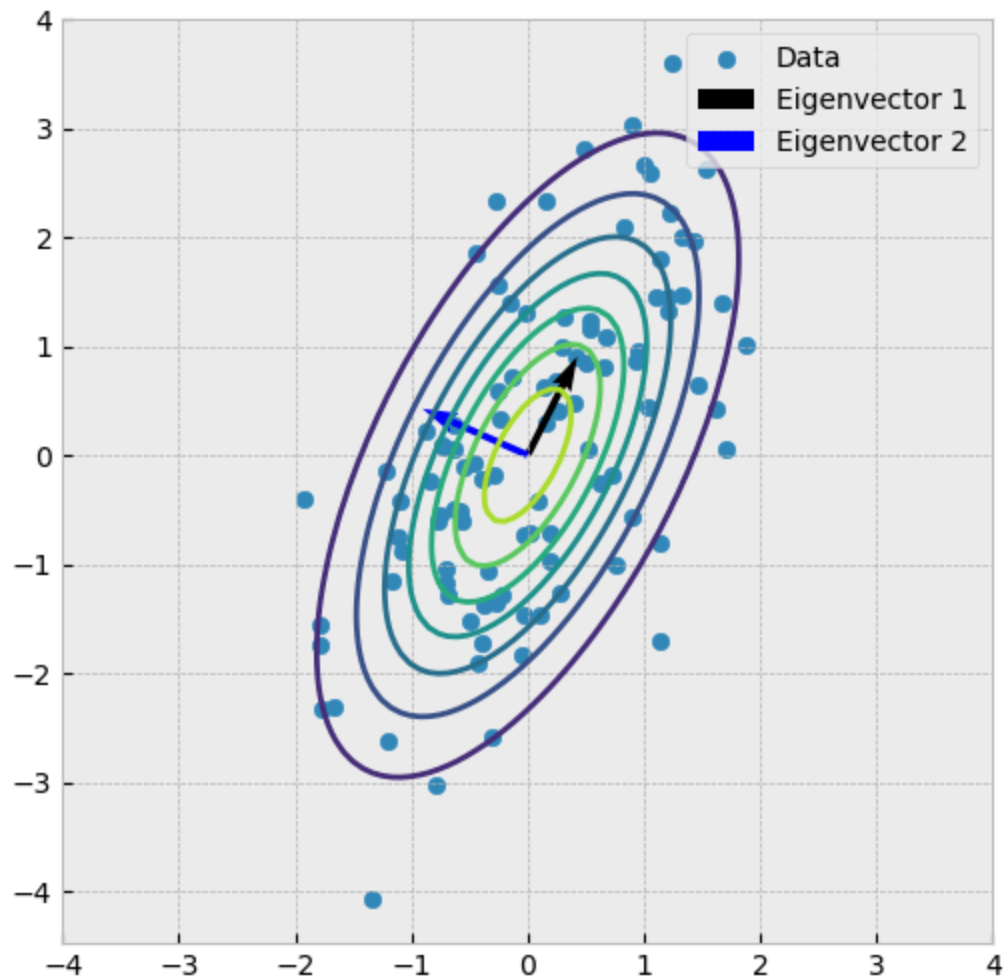
```
L=L[::-1]
```

```
V=V[:,::-1]
```

```
L, V
```

```
Out[8]: (array([2.53154708, 0.42919357]),
        array([[ 0.4257714 , -0.90483076],
               [ 0.90483076,  0.4257714 ]]))
```

```
In [9]: plot_contours(K)
plt.scatter(data[:,0],data[:,1])
plotvec(V[:,0], V[:,1])
plt.legend(['Data', 'Eigenvector 1', 'Eigenvector 2']);
```

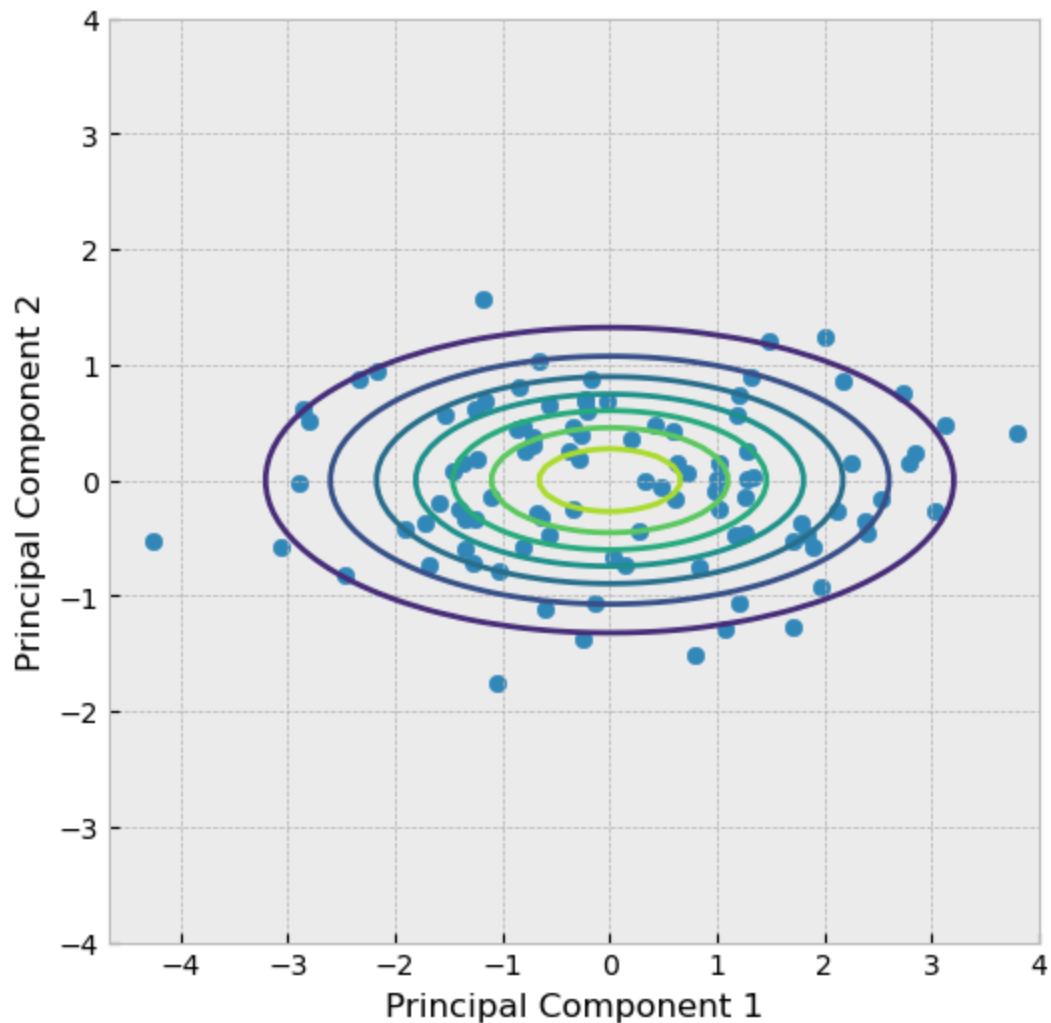
```
In [10]: rotated = V.T@data.T
rotated.shape
```

```
Out[10]: (2, 100)
```

```
In [11]: K2 = np.cov(rotated)
np.round(K2,7)
```

```
Out[11]: array([[2.5315471, 0.        ],
                [0.        , 0.4291936]])
```

```
In [12]: plot_contours(K2)
plt.scatter(rotated[0,:],rotated[1,:])
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2');
```



```
In [13]: # Rotation with respect to [1,0]
e1=np.array([[1],[0]])

angle = np.degrees(np.arccos((e1.T@V[:,0])/(np.linalg.norm(e1)*np.linalg.norm(V[:,0])))
angle
```

```
Out[13]: 64.80049943355938
```

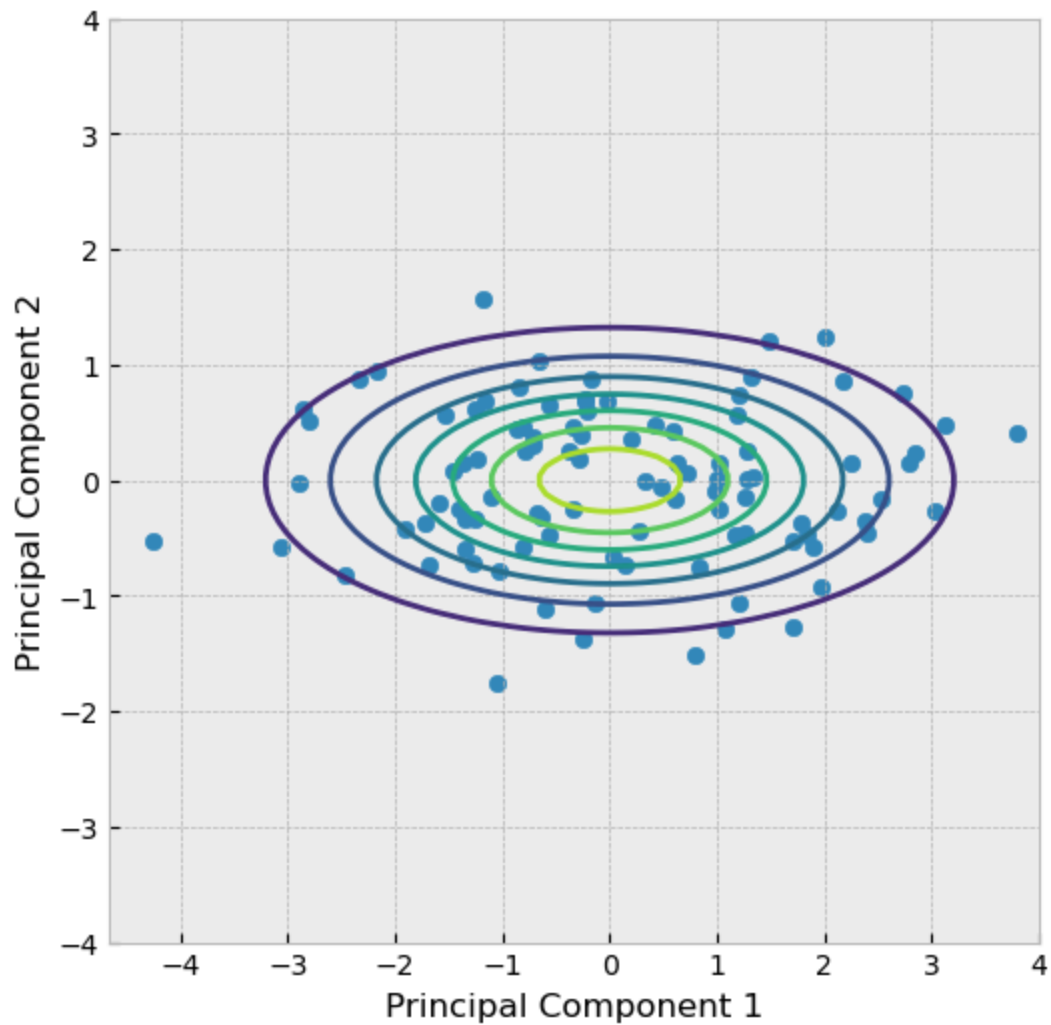
```
In [14]: rotated2 = makerot(-angle)@data.T
```

```
In [15]: K3 = np.cov(rotated2)

np.round(K3,7)
```

```
Out[15]: array([[2.5315471, 0.          ],
               [0.          , 0.4291936]])
```

```
In [16]: plot_contours(K3)
plt.scatter(rotated2[0,:],rotated2[1,:])
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2');
```



Wine Dataset

```
In [17]: df_wine = pd.read_csv('https://archive.ics.uci.edu/ml/machine-learning-databases/wine/wine.data')

df_wine.columns = ['Class label', 'Alcohol',
                   'Malic acid', 'Ash',
                   'Alcalinity of ash', 'Magnesium',
                   'Total phenols', 'Flavanoids',
                   'Nonflavanoid phenols',
                   'Proanthocyanins',
                   'Color intensity', 'Hue',
                   'OD280/OD315 of diluted wines',
                   'Proline']

df_wine
```

Out[17]:

	Class label	Alcohol	Malic acid	Ash	Alcalinity of ash	Magnesium	Total phenols	Flavanoids	Nonflav ph
0	1	14.23	1.71	2.43	15.6	127	2.80	3.06	
1	1	13.20	1.78	2.14	11.2	100	2.65	2.76	
2	1	13.16	2.36	2.67	18.6	101	2.80	3.24	
3	1	14.37	1.95	2.50	16.8	113	3.85	3.49	
4	1	13.24	2.59	2.87	21.0	118	2.80	2.69	
...	...	...	...	...	...	...	...	...	
173	3	13.71	5.65	2.45	20.5	95	1.68	0.61	
174	3	13.40	3.91	2.48	23.0	102	1.80	0.75	
175	3	13.27	4.28	2.26	20.0	120	1.59	0.69	
176	3	13.17	2.59	2.37	20.0	120	1.65	0.68	
177	3	14.13	4.10	2.74	24.5	96	2.05	0.76	

178 rows x 14 columns

```
In [18]: from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Target Labels
t = df_wine['Class label'].values

# Feature Matrix
X = df_wine.drop(['Class label'], axis=1).values
print(X.shape)

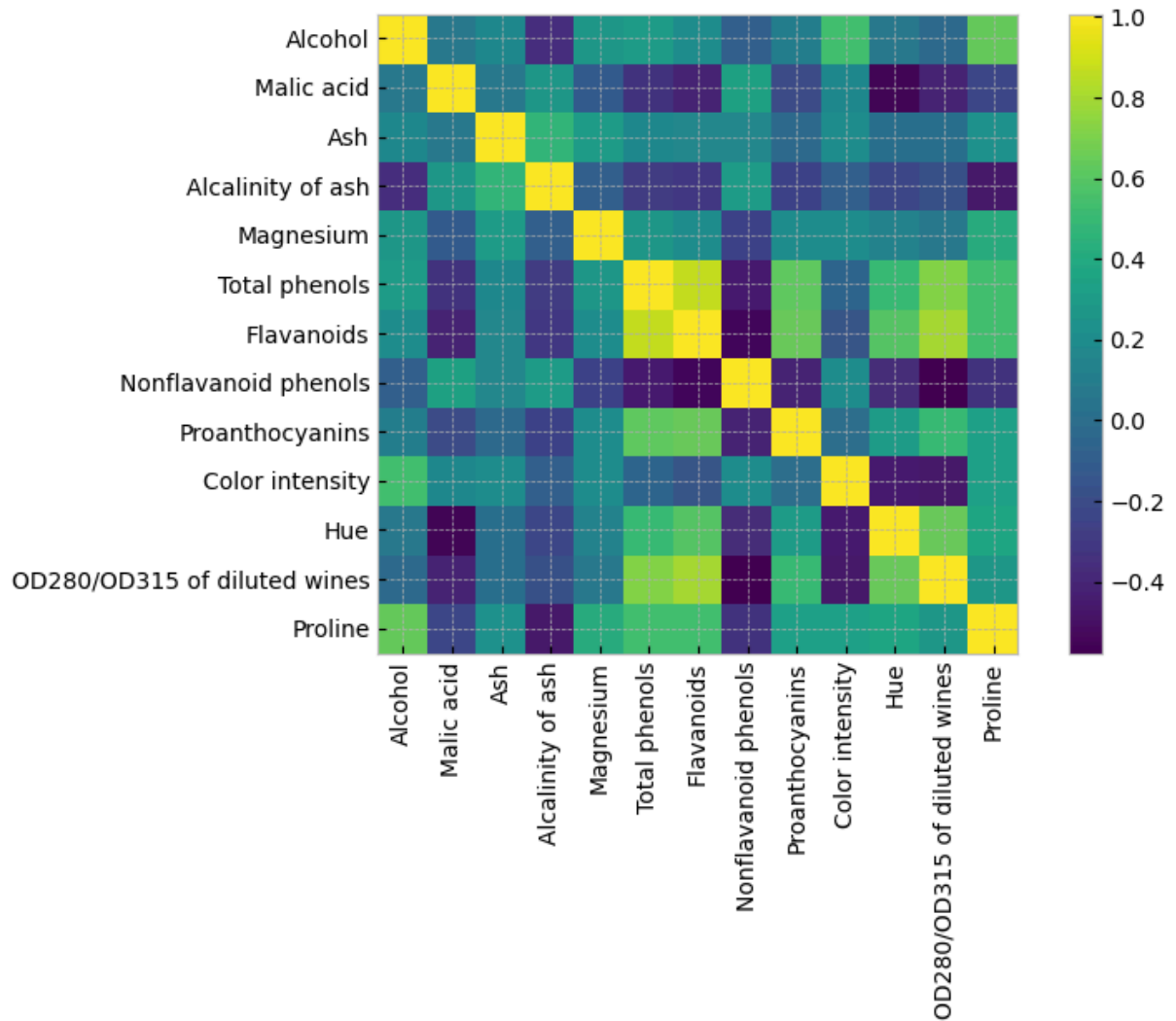
# Stratified partition of the data into training/test sets
X_train, X_test, t_train, t_test = train_test_split(X, t,
                                                    test_size=0.3,
                                                    stratify=t,
                                                    random_state=0)

# Scaling data
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

(178, 13)

```
In [19]: # Covariance dataset
cov_train = np.cov(X_train.T)

plt.figure(figsize=(8,5))
plt.imshow(cov_train)
plt.colorbar()
plt.xticks(range(13),df_wine.columns[1:],rotation=90)
plt.yticks(range(13),df_wine.columns[1:]);
```



Building a function to implement PCA from scratch:

```
In [27]: def myPCA(X, m, display=1):
    '''This function implements PCA. The data matrix X is DxN matrix,
    where D is the dimension and N the number of points'''

    D,N = X.shape

    # Covariance of the input data X
    cov_mat = np.cov(X)

    # Find eigenvectors and eigenvalues
    eigen_vals, eigen_vecs = np.linalg.eigh(cov_mat)
    # np.linalg.eigh sorts vectors in increasing order of eigenvalues

    # Sort eigenvectors by magnitude of eigenvalues
    L = eigen_vals[::-1]
    U = eigen_vecs[:,::-1]

    # Linear transformation
    A = U[:, :m].T
    # Eigenvectors stored in columns
```

```

#compute explained variance and plot
sum_eigs = np.sum(L) # summing all eigenvalues
explained_var = np.array([(i/sum_eigs) for i in L]) # explained variance
cumulative_explained_var = np.cumsum(explained_var) # cumulative explained variance

# Visualizing explained variance for each PC and cumulative
if display:
    plt.bar(range(1,D+1), explained_var, alpha=0.5, align='center', label='individual explained variance')
    plt.step(range(1,D+1), cumulative_explained_var, alpha=0.5, where='right', label='cumulative explained variance')
    plt.ylabel('Explained variance ratio')
    plt.xlabel('Principal components')
    plt.legend(loc='best');

return A, explained_var

```

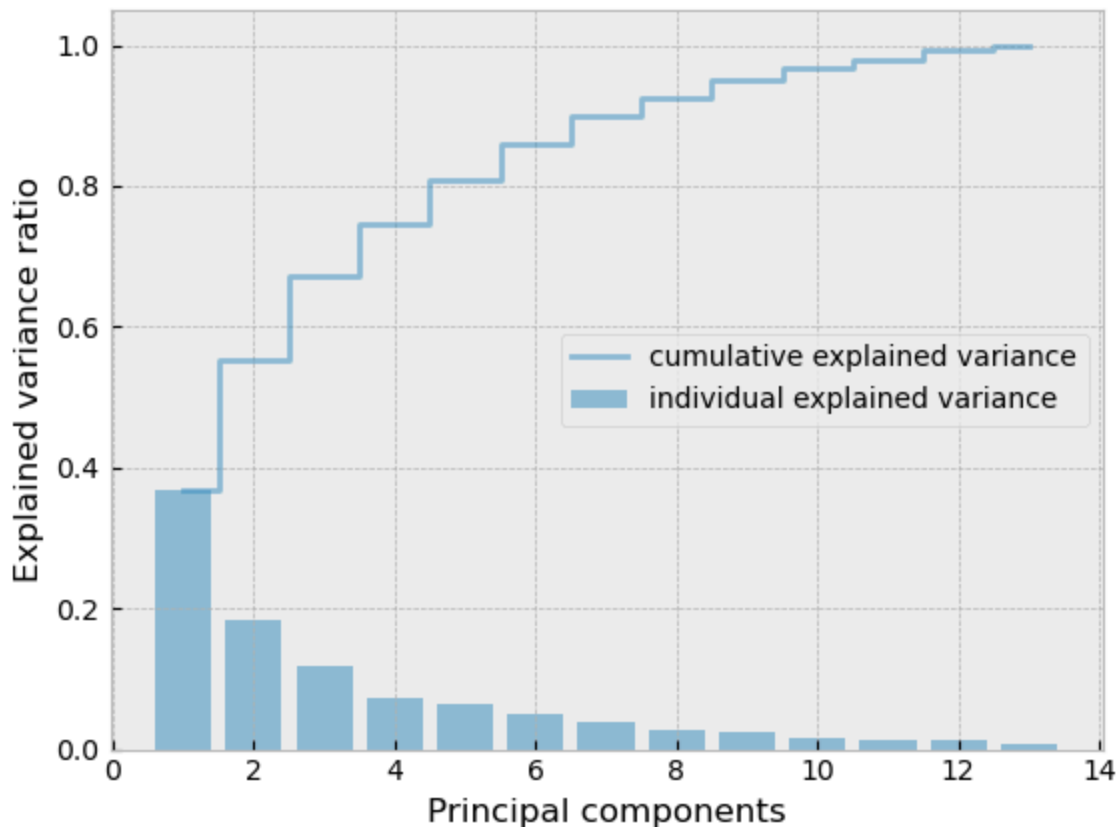
In [28]: `X_train.shape`

Out[28]: (124, 13)

In [29]: `A, explained_var = myPCA(X_train.T, 13)`

`A.shape`

Out[29]: (13, 13)



In [30]: `explained_var`

```
Out[30]: array([0.36951469, 0.18434927, 0.11815159, 0.07334252, 0.06422108,
               0.05051724, 0.03954654, 0.02643918, 0.02389319, 0.01629614,
               0.01380021, 0.01172226, 0.00820609])
```

```
In [31]: np.where(np.cumsum(explained_var)>=0.9)
```

```
Out[31]: (array([ 7,  8,  9, 10, 11, 12]),)
```

```
In [32]: np.cumsum(explained_var)[7]
```

```
# So, we need at least 8 eigenvectors to explain at least 90% of the variance
```

```
Out[32]: 0.92608211032677
```

The resulting plot indicates that the first principal component alone accounts for 40 percent of the variance. Also, we can see that the first two principal components combined explain almost 60 percent of the variance in the data.

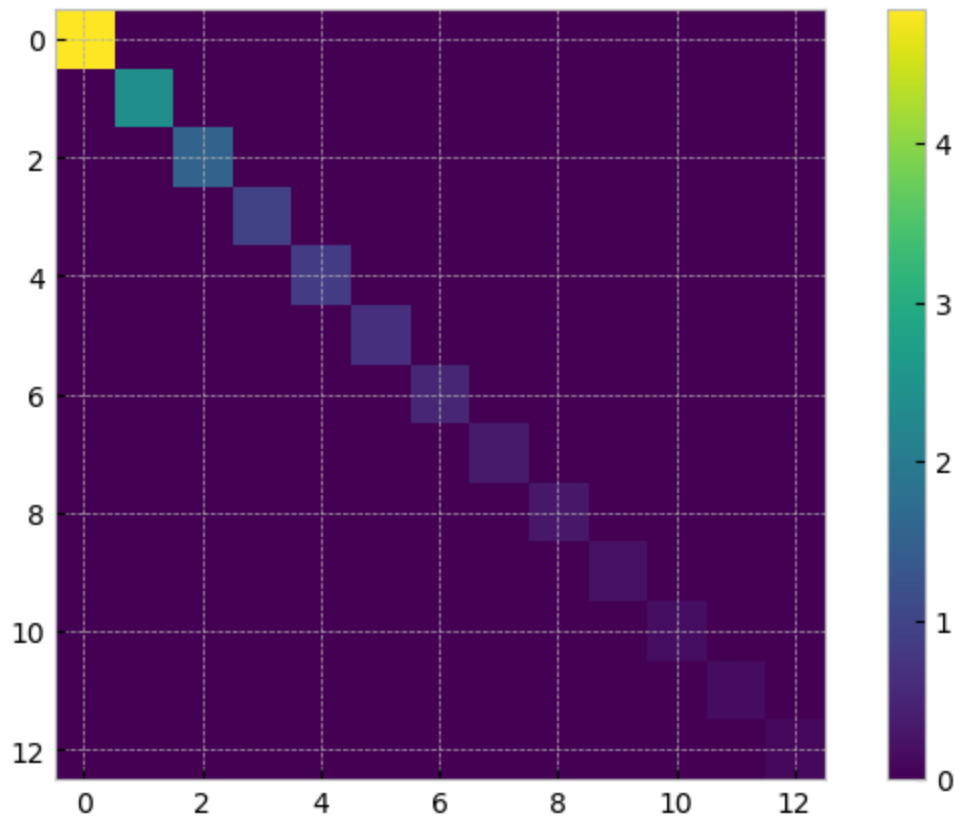
Although the explained variance plot reminds us of the feature importance, we shall remind ourselves that PCA is an unsupervised method, which means that information about the class labels is ignored.

```
In [33]: X_train_pca = (A@X_train.T).T
         print(X_train.shape, X_train_pca.shape)
```

```
(124, 13) (124, 13)
```

```
In [34]: # Check
         cov_mat = np.cov(X_train_pca.T)

         plt.figure(figsize=(8,5))
         plt.imshow(cov_mat)
         plt.colorbar();
```



PCA with `scikit-learn`

```
In [35]: from sklearn.decomposition import PCA
```

```
In [36]: # PCA?
```

```
In [37]: pca = PCA(n_components=13)
pca
```

```
Out[37]: PCA
PCA(n_components=13)
```

```
In [38]: pca.fit(X_train) # input expects a matrix of size NxD
```

```
Out[38]: PCA
PCA(n_components=13)
```

```
In [39]: pca.explained_variance_ratio_
```

```
Out[39]: array([0.36951469, 0.18434927, 0.11815159, 0.07334252, 0.06422108,
                0.05051724, 0.03954654, 0.02643918, 0.02389319, 0.01629614,
                0.01380021, 0.01172226, 0.00820609])
```



```
In [40]: np.cumsum(pca.explained_variance_ratio_)
```

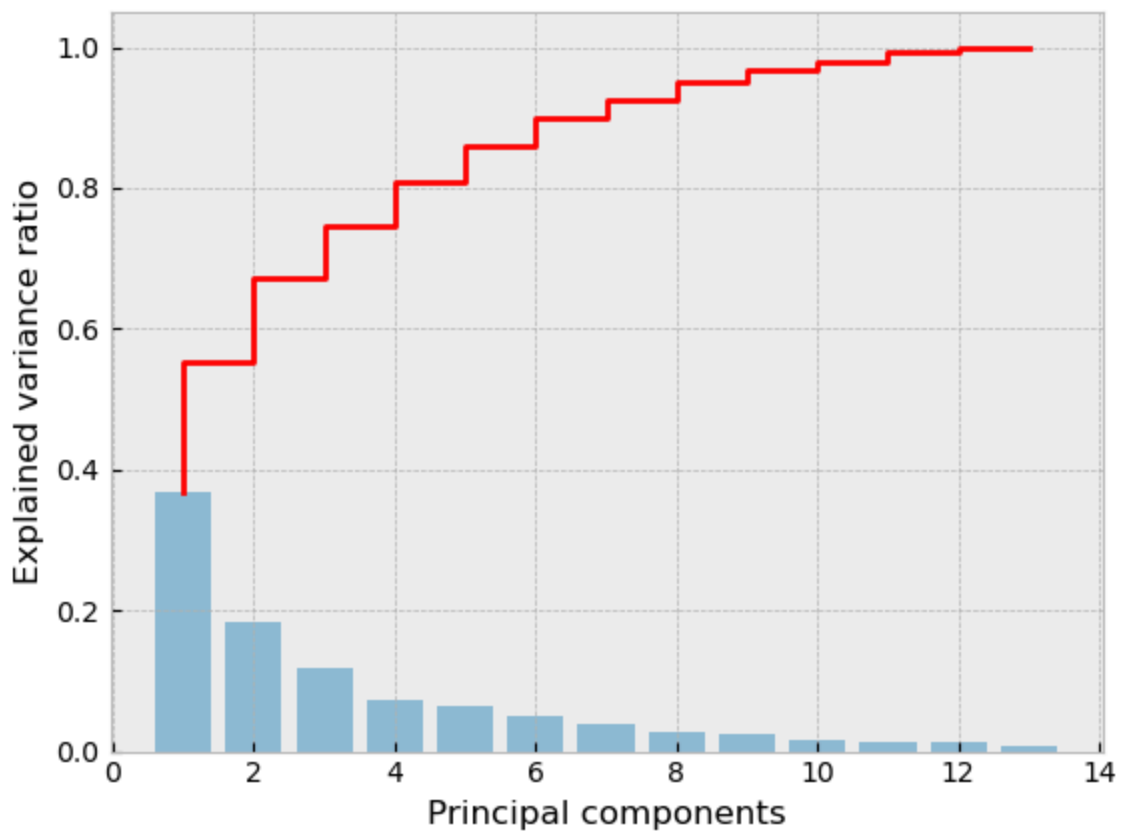
```
Out[40]: array([0.36951469, 0.55386396, 0.67201555, 0.74535807, 0.80957914,
                0.86009639, 0.89964293, 0.92608211, 0.9499753 , 0.96627144,
                0.98007165, 0.99179391, 1.          ])
```

```
In [41]: # The matrix A = U.T is
```

```
pca.components_.shape
```

```
Out[41]: (13, 13)
```

```
In [42]: plt.step(range(1,14),np.cumsum(pca.explained_variance_ratio_),c='r')
plt.bar(range(1,14),pca.explained_variance_ratio_, alpha=0.5)
plt.ylabel('Explained variance ratio')
plt.xlabel('Principal components');
```



```
In [43]: y_train_pca = pca.transform(X_train)
```

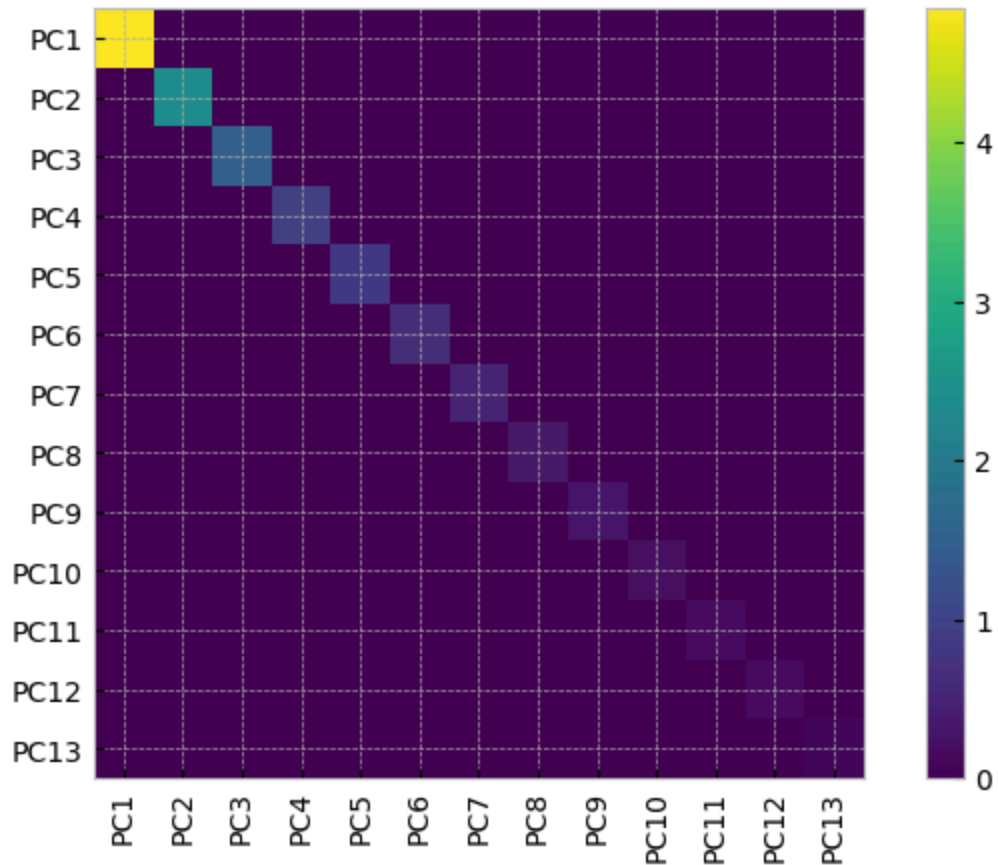
```
y_train_pca.shape
```

```
Out[43]: (124, 13)
```

```
In [44]: cov_mat = np.cov(y_train_pca.T)
```

```
plt.figure(figsize=(8,5))
plt.imshow(cov_mat)
plt.colorbar()
```

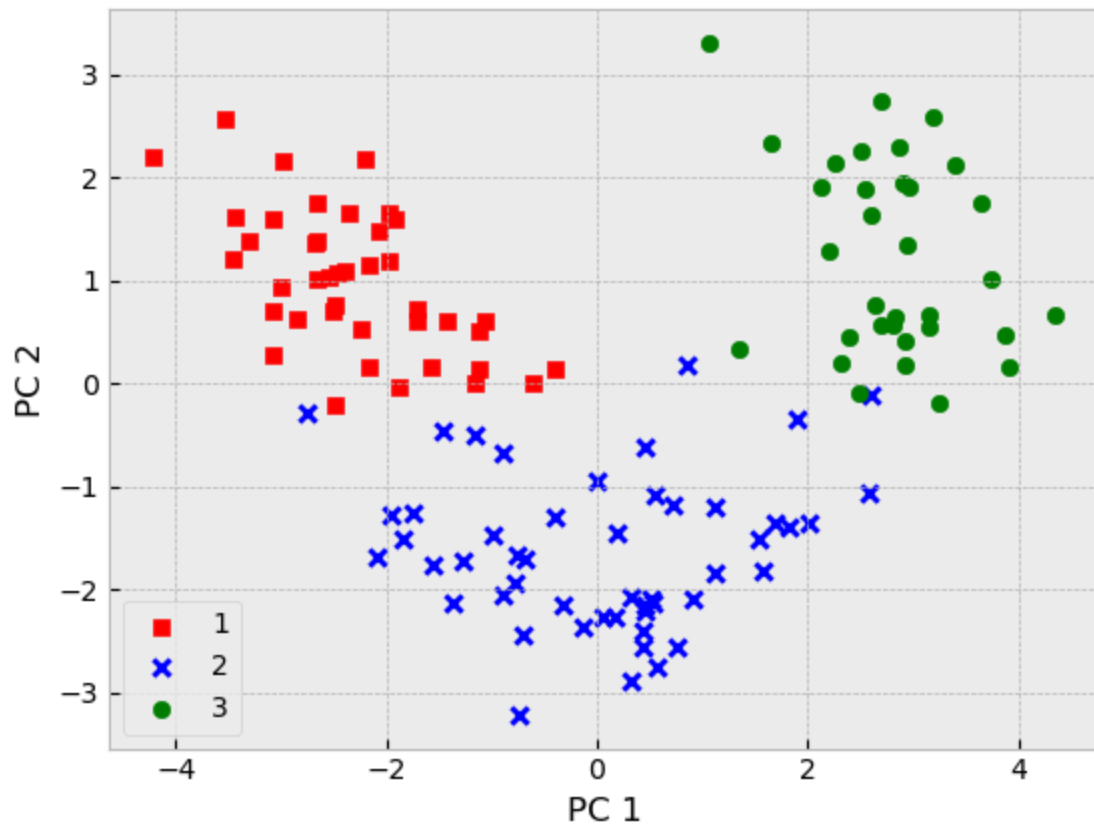
```
plt.yticks(range(13), ['PC'+str(i) for i in range(1,14)])
plt.xticks(range(13), ['PC'+str(i) for i in range(1,14)], rotation=90);
```



```
In [45]: colors = ['r', 'b', 'g']
markers = ['s', 'x', 'o']

for l, c, m in zip(np.unique(t_train), colors, markers):
    plt.scatter(y_train_pca[t_train==l, 0], y_train_pca[t_train==l, 1], c=c,
               marker=m)

plt.xlabel('PC 1')
plt.ylabel('PC 2')
plt.legend(loc='lower left')
plt.show()
```



The training data is used to find the new features (eigenvectors). We can then represent the test set in this new feature space:

```
In [46]: y_test_pca = pca.transform(X_test)
y_test_pca.shape
```

```
Out[46]: (54, 13)
```

Applications: Image Compression (Eigenfaces)

```
In [47]: from sklearn.datasets import fetch_olivetti_faces
faces = fetch_olivetti_faces(return_X_y=False)
print(faces.DESCR)
```

```
.. _olivetti_faces_dataset:
```

The Olivetti faces dataset

`This dataset contains a set of face images`_ taken between April 1992 and April 1994 at AT&T Laboratories Cambridge. The
:func:`sklearn.datasets.fetch\_olivetti\_faces` function is the data
fetching / caching function that downloads the data
archive from AT&T.

.. _This dataset contains a set of face images: <https://cam-orl.co.uk/facedatabase.html>

As described on the original website:

There are ten different images of each of 40 distinct subjects. For some subjects, the images were taken at different times, varying the lighting, facial expressions (open / closed eyes, smiling / not smiling) and facial details (glasses / no glasses). All the images were taken against a dark homogeneous background with the subjects in an upright, frontal position (with tolerance for some side movement).

****Data Set Characteristics:****

Classes	40
Samples total	400
Dimensionality	4096
Features	real, between 0 and 1

The image is quantized to 256 grey levels and stored as unsigned 8-bit integers; the loader will convert these to floating point values on the interval [0, 1], which are easier to work with for many algorithms.

The "target" for this database is an integer from 0 to 39 indicating the identity of the person pictured; however, with only 10 examples per class, this relatively small dataset is more interesting from an unsupervised or semi-supervised perspective.

The original dataset consisted of 92 x 112, while the version available here consists of 64x64 images.

When using these images, please give credit to AT&T Laboratories Cambridge.

```
In [48]: X = faces.data # data matrix  
  
         t = faces.target # target label  
  
         X.shape, t.shape # 400 images, each of size 64x64=4096 pixels
```



```
In [51]: from sklearn.model_selection import train_test_split

X_train, X_test, t_train, t_test = train_test_split(X, t,
                                                    test_size=0.2,
                                                    stratify=t,
                                                    random_state=0)

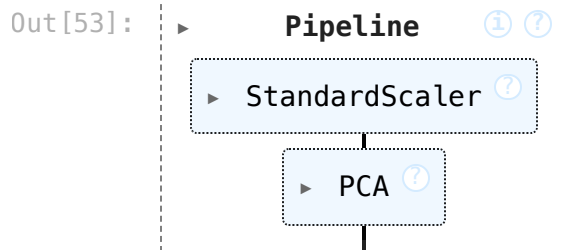
X_train.shape, t_train.shape, X_test.shape, t_test.shape
```

```
Out[51]: ((320, 4096), (320,), (80, 4096), (80,))
```

```
In [53]: from sklearn.pipeline import Pipeline

pipe = Pipeline([('scaler', StandardScaler()),
                 ('pca', PCA(n_components=320))])

pipe.fit(X_train, t_train)
```



The maximum number of dimensions using PCA is $\min(D, N)$.

```
In [54]: pipe.named_steps.pca.explained_variance_ratio_
```

```
Out[54]: array([2.64776915e-01, 1.23178974e-01, 7.80273974e-02, 4.80991565e-02,
3.29087637e-02, 3.09590604e-02, 2.45285109e-02, 2.08553821e-02,
1.83627233e-02, 1.73735749e-02, 1.55731961e-02, 1.38689019e-02,
1.25643350e-02, 1.17299752e-02, 1.12679508e-02, 9.63340886e-03,
8.99304263e-03, 8.25929735e-03, 7.93462712e-03, 7.45459273e-03,
6.87649194e-03, 6.53506862e-03, 6.06589345e-03, 5.63047128e-03,
5.47134411e-03, 5.34940977e-03, 5.28507913e-03, 4.72592935e-03,
4.36570495e-03, 4.30482812e-03, 4.12363326e-03, 3.95362824e-03,
3.79256927e-03, 3.42833274e-03, 3.39972507e-03, 3.30448127e-03,
3.21943033e-03, 3.00289085e-03, 2.92331167e-03, 2.86730751e-03,
2.72023375e-03, 2.66562682e-03, 2.63113575e-03, 2.55247368e-03,
2.48611765e-03, 2.36396794e-03, 2.35044188e-03, 2.19845329e-03,
2.15451443e-03, 2.11767526e-03, 2.06505880e-03, 2.04270729e-03,
1.93845190e-03, 1.91345869e-03, 1.86564191e-03, 1.82843127e-03,
1.78010506e-03, 1.69170764e-03, 1.64255232e-03, 1.63180940e-03,
1.59568142e-03, 1.54116529e-03, 1.50381203e-03, 1.48862612e-03,
1.45757792e-03, 1.41013612e-03, 1.40030275e-03, 1.36304938e-03,
1.33758073e-03, 1.31597812e-03, 1.30924734e-03, 1.27956388e-03,
1.26308110e-03, 1.20210124e-03, 1.19595858e-03, 1.16177194e-03,
1.15349854e-03, 1.13067019e-03, 1.11895555e-03, 1.08639454e-03,
1.07471400e-03, 1.06055872e-03, 1.04618748e-03, 1.00335618e-03,
9.99348937e-04, 9.70728754e-04, 9.65471147e-04, 9.34082782e-04,
9.24381835e-04, 9.12214338e-04, 8.94884754e-04, 8.90700670e-04,
8.82987399e-04, 8.64600530e-04, 8.56253144e-04, 8.32938938e-04,
8.18177476e-04, 8.08309065e-04, 7.85373501e-04, 7.84574251e-04,
7.76891713e-04, 7.69451028e-04, 7.51753047e-04, 7.35329930e-04,
7.28080107e-04, 7.17355812e-04, 7.04922888e-04, 6.97856885e-04,
6.83418941e-04, 6.72494119e-04, 6.65342726e-04, 6.63583283e-04,
6.50054659e-04, 6.38029655e-04, 6.35952456e-04, 6.26216293e-04,
6.12769567e-04, 6.10554591e-04, 5.97412058e-04, 5.86141425e-04,
5.78780193e-04, 5.68919233e-04, 5.68156771e-04, 5.56253712e-04,
5.52029931e-04, 5.47082396e-04, 5.40642650e-04, 5.34346153e-04,
5.21279813e-04, 5.14646468e-04, 5.08267956e-04, 5.00100257e-04,
4.94214415e-04, 4.91135812e-04, 4.83596581e-04, 4.77980066e-04,
4.74324188e-04, 4.64646262e-04, 4.60114738e-04, 4.52576176e-04,
4.50138818e-04, 4.44460835e-04, 4.42803983e-04, 4.34433081e-04,
4.27916995e-04, 4.21577948e-04, 4.15236398e-04, 4.11446148e-04,
4.07369371e-04, 4.02911071e-04, 3.97890166e-04, 3.96325835e-04,
3.90322675e-04, 3.88663728e-04, 3.79182311e-04, 3.78056284e-04,
3.77626595e-04, 3.74953379e-04, 3.70742957e-04, 3.67629342e-04,
3.61151935e-04, 3.53813026e-04, 3.49846960e-04, 3.42484767e-04,
3.39594088e-04, 3.38856742e-04, 3.33108066e-04, 3.30312207e-04,
3.26793961e-04, 3.22648848e-04, 3.19644751e-04, 3.15503130e-04,
3.13336728e-04, 3.09653609e-04, 3.03714594e-04, 3.00713931e-04,
2.99754378e-04, 2.97153369e-04, 2.92162120e-04, 2.86817900e-04,
2.83566100e-04, 2.79211759e-04, 2.77451618e-04, 2.73123238e-04,
2.70135701e-04, 2.65932293e-04, 2.63318361e-04, 2.60631874e-04,
2.59404333e-04, 2.56821717e-04, 2.53525330e-04, 2.51362188e-04,
2.45656993e-04, 2.44599301e-04, 2.41141912e-04, 2.39348024e-04,
2.37254877e-04, 2.33831393e-04, 2.32361577e-04, 2.29851066e-04,
2.27975659e-04, 2.27614713e-04, 2.21163136e-04, 2.18929243e-04,
2.16946180e-04, 2.13538253e-04, 2.11385966e-04, 2.07684177e-04,
2.05887176e-04, 2.03660005e-04, 2.02888390e-04, 1.99725953e-04,
1.96947571e-04, 1.95763743e-04, 1.94470878e-04, 1.93139393e-04,
1.89346552e-04, 1.85328347e-04, 1.83648925e-04, 1.82690594e-04,
1.80248855e-04, 1.79671173e-04, 1.77515569e-04, 1.74909874e-04,
```

```

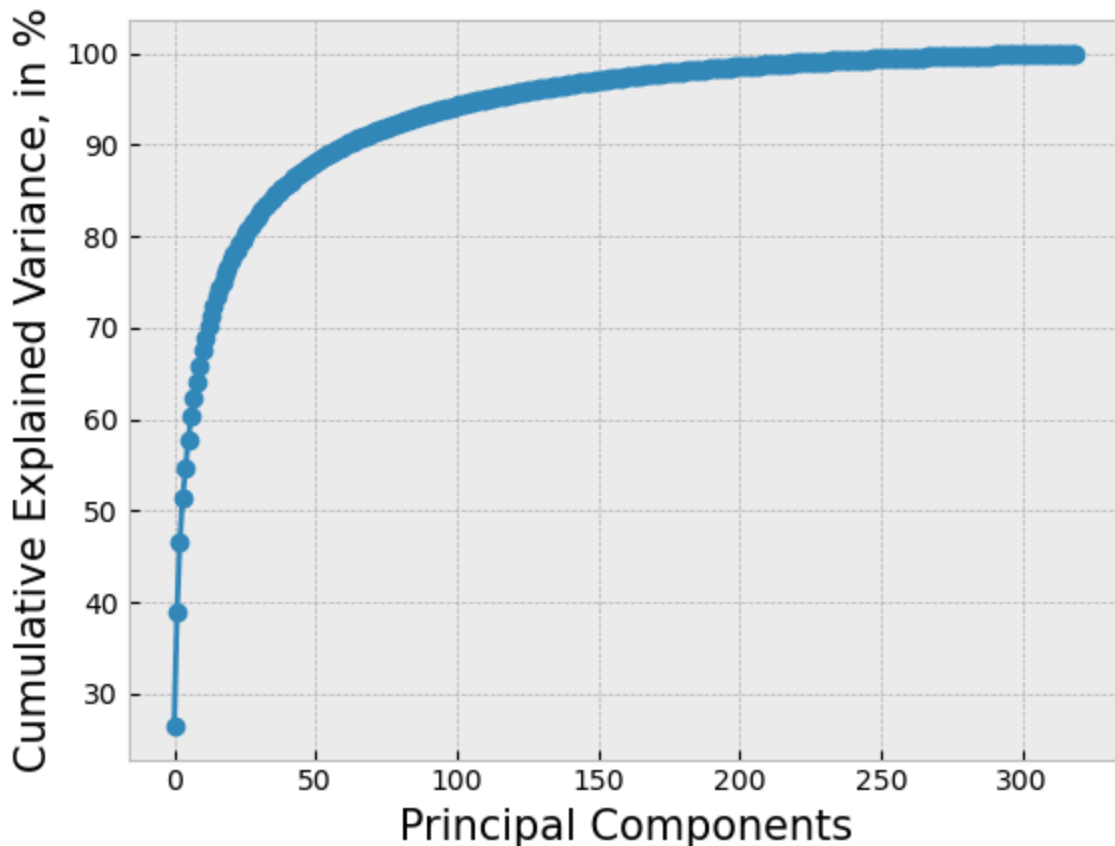
1.73072243e-04, 1.72181593e-04, 1.69760096e-04, 1.66444166e-04,
1.65311430e-04, 1.63872799e-04, 1.63312769e-04, 1.58542025e-04,
1.56540773e-04, 1.55112997e-04, 1.52656212e-04, 1.52171095e-04,
1.49810148e-04, 1.48097504e-04, 1.46036342e-04, 1.44790378e-04,
1.43387238e-04, 1.42207165e-04, 1.41745419e-04, 1.39800904e-04,
1.37413459e-04, 1.36431248e-04, 1.35209659e-04, 1.34767586e-04,
1.32500863e-04, 1.30463319e-04, 1.27640873e-04, 1.25214196e-04,
1.22062120e-04, 1.21624355e-04, 1.19785429e-04, 1.18249452e-04,
1.16901014e-04, 1.15672017e-04, 1.14278700e-04, 1.13608279e-04,
1.11532237e-04, 1.10898916e-04, 1.10247158e-04, 1.08488719e-04,
1.07896863e-04, 1.05828425e-04, 1.04434228e-04, 1.03746235e-04,
1.00778241e-04, 9.90969638e-05, 9.87607564e-05, 9.76025403e-05,
9.63385464e-05, 9.49906680e-05, 9.42373808e-05, 9.31835093e-05,
9.23864645e-05, 9.13397307e-05, 8.99041770e-05, 8.88103896e-05,
8.80668013e-05, 8.65803449e-05, 8.44478200e-05, 8.24943854e-05,
8.19894194e-05, 8.09690173e-05, 7.84080839e-05, 7.72073254e-05,
7.61409246e-05, 7.54717548e-05, 7.47223094e-05, 7.38757080e-05,
7.26210856e-05, 7.00742748e-05, 6.97929936e-05, 6.78478318e-05,
6.75645060e-05, 6.65270127e-05, 6.59094876e-05, 6.43663516e-05,
6.21122017e-05, 6.00726707e-05, 5.96964383e-05, 5.90088566e-05,
5.77408173e-05, 5.52833517e-05, 5.42874150e-05, 5.33950988e-05,
5.15487918e-05, 5.03246993e-05, 4.97236178e-05, 4.89956255e-05,
4.65261619e-05, 4.53684552e-05, 4.26441111e-05, 3.59411315e-05,
3.45008302e-05, 3.24013854e-05, 2.81656849e-05, 1.06691635e-15],
dtype=float32)

```

```

In [55]: plt.plot(100*np.cumsum(pipe.named_steps.pca.explained_variance_ratio_), '-o')
plt.xlabel('Principal Components',size=15)
plt.ylabel('Cumulative Explained Variance, in %', size=15);

```




```
In [56]: np.where(np.cumsum(pipe.named_steps.pca.explained_variance_ratio_)>=0.9)
```

```
Out[56]: (array([ 61,  62,  63,  64,  65,  66,  67,  68,  69,  70,  71,  72,  73,
                  74,  75,  76,  77,  78,  79,  80,  81,  82,  83,  84,  85,  86,
                  87,  88,  89,  90,  91,  92,  93,  94,  95,  96,  97,  98,  99,
                100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112,
                113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125,
                126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138,
                139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151,
                152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164,
                165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177,
                178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190,
                191, 192, 193, 194, 195, 196, 197, 198, 199, 200, 201, 202, 203,
                204, 205, 206, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216,
                217, 218, 219, 220, 221, 222, 223, 224, 225, 226, 227, 228, 229,
                230, 231, 232, 233, 234, 235, 236, 237, 238, 239, 240, 241, 242,
                243, 244, 245, 246, 247, 248, 249, 250, 251, 252, 253, 254, 255,
                256, 257, 258, 259, 260, 261, 262, 263, 264, 265, 266, 267, 268,
                269, 270, 271, 272, 273, 274, 275, 276, 277, 278, 279, 280, 281,
                282, 283, 284, 285, 286, 287, 288, 289, 290, 291, 292, 293, 294,
                295, 296, 297, 298, 299, 300, 301, 302, 303, 304, 305, 306, 307,
                308, 309, 310, 311, 312, 313, 314, 315, 316, 317, 318, 319]),)
```

```
In [57]: np.cumsum(pipe.named_steps.pca.explained_variance_ratio_)[61]
```

```
Out[57]: 0.9007526
```

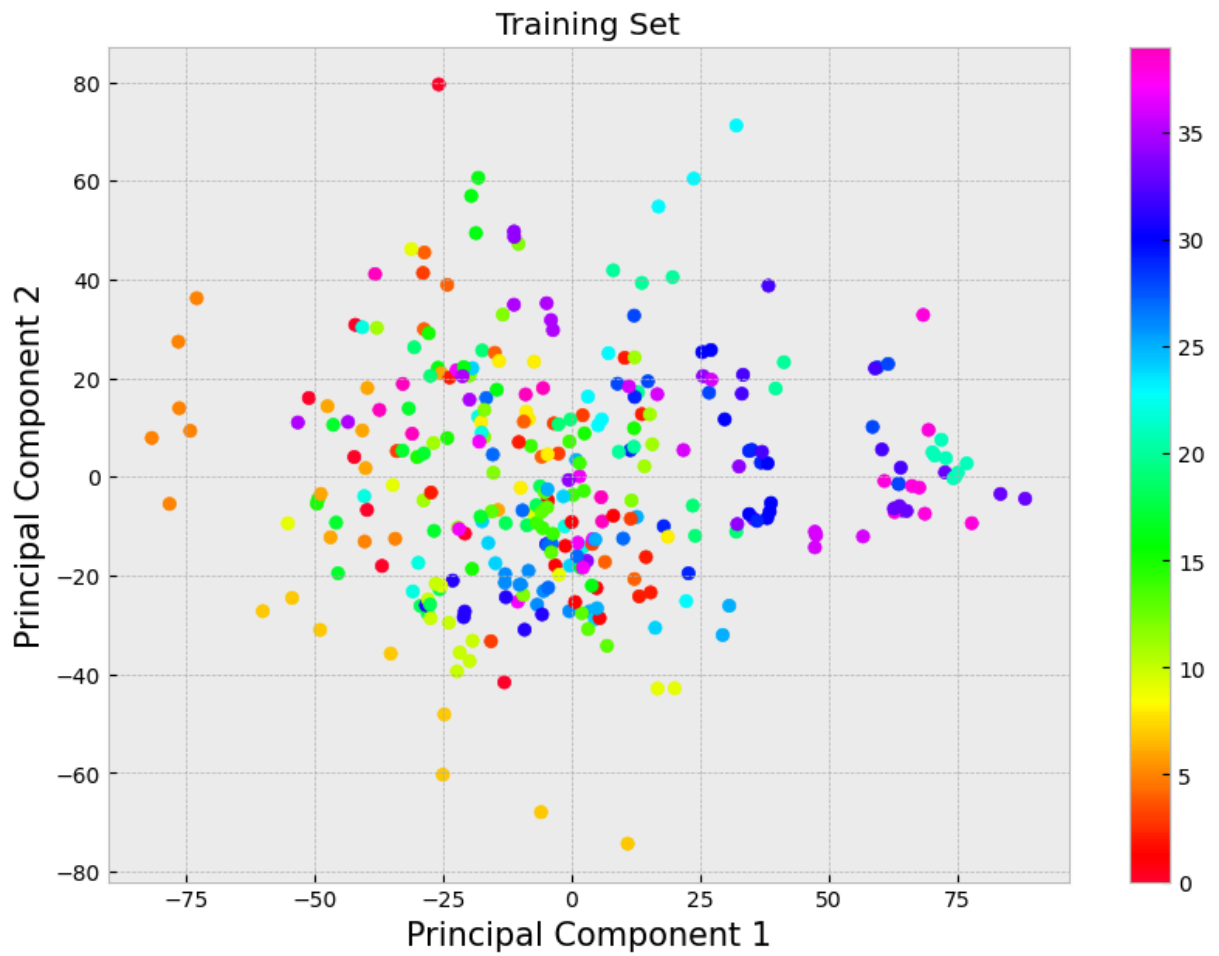
In order to explain 90% of the variance in the data, we need to preserve 63 principal components.

Let's project to 2-D so we can plot it:

```
In [59]: ypca = pipe.transform(X_train)
ypca.shape #320 samples by 320 dimensions
```

```
Out[59]: (320, 320)
```

```
In [60]: plt.figure(figsize=(10,7))
plt.scatter(ypca[:,0], ypca[:,1], c=t_train, cmap=plt.cm.gist_rainbow)
plt.xlabel('Principal Component 1', size=15)
plt.ylabel('Principal Component 2', size=15)
plt.title('Training Set')
plt.colorbar();
```



Not that the 40 classes are overlapping in the linear projection space. This is because PCA is **unsupervised**, it does use the class labels *anywhere* in finding the matrix for linear projection.

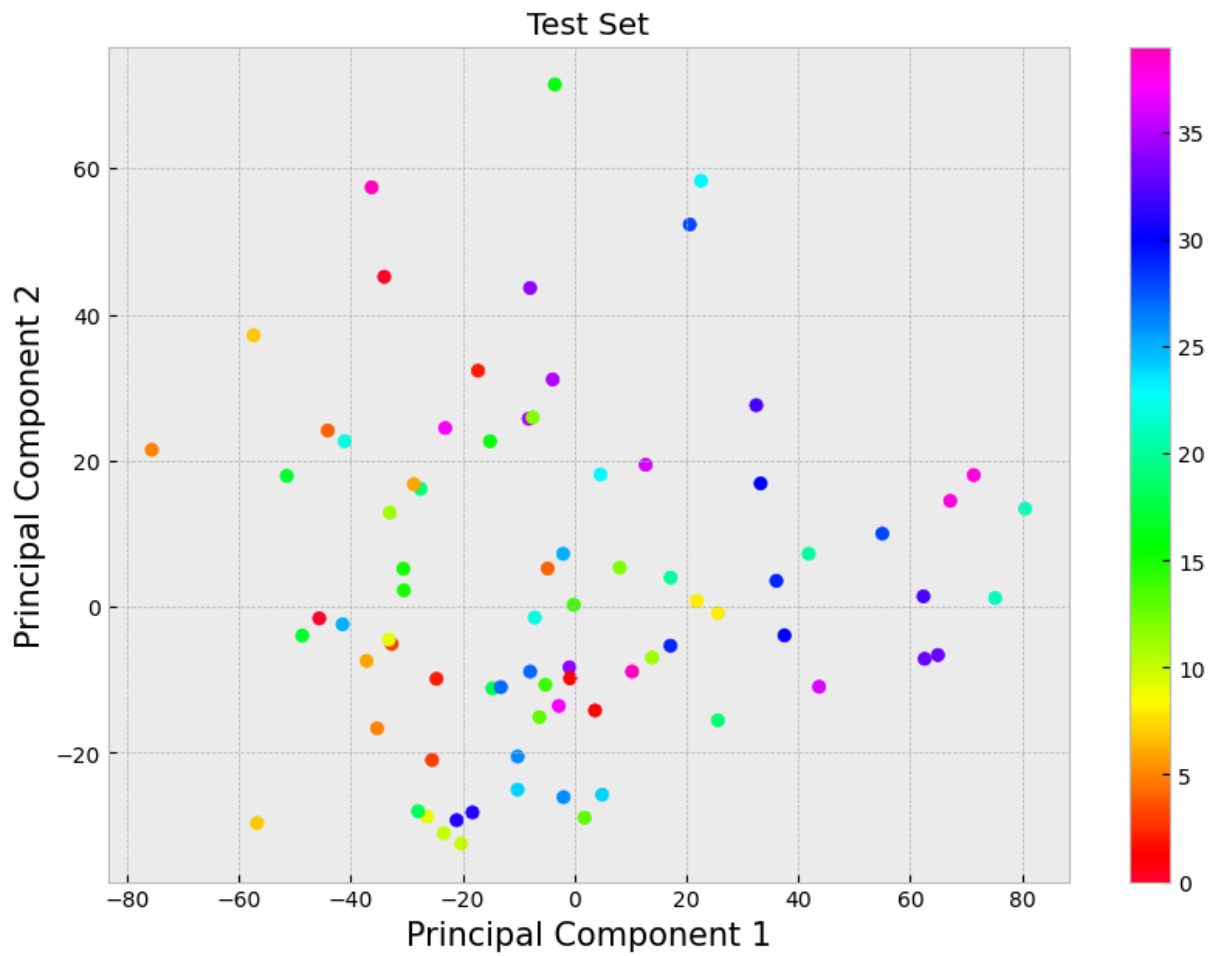
To apply this transformation in the test set, simply multiply the resultant modal matrix with the scaled test set:

```
In [63]: # Transform the test set using the linear transformation found with the train set
ypca_test = pipe.transform(X_test)

ypca_test.shape
```

```
Out[63]: (80, 320)
```

```
In [62]: plt.figure(figsize=(10,7))
plt.scatter(ypca_test[:,0], ypca_test[:,1], c=t_test, cmap=plt.cm.gist_rainbow)
plt.xlabel('Principal Component 1', size=15)
plt.ylabel('Principal Component 2', size=15)
plt.title('Test Set')
plt.colorbar();
```



You can access the linear transformation $\mathbf{A} = \mathbf{U}^T$ using the method `components_`:

```
In [64]: A = pipe.named_steps.pca.components_  
A.shape
```

```
Out[64]: (320, 4096)
```

Note that the eigenvectors are described in the original space, that is, they are 4096-dimensional!

Since we are working with images, we can reshape them back to a 64×64 image and see what are the regions in the image with maximum explained variance! This is called the **eigenfaces**.

```
In [65]: n_components = 16  
  
fig=plt.figure(figsize=(10,10))  
for i in range(n_components):  
    fig.add_subplot(4,4,i+1)  
    plt.imshow(abs(A[i,:].reshape(64,64)),cmap='gray')  
    plt.axis('off')
```



The eigenvectors are describing the pixels in the 64x64 image that explain the most variance. The more eigenvectors are kept, the better the reconstructed image is.

Since the projection is given by:

$$\mathbf{Y} = \mathbf{A}\mathbf{X}$$

In order to recover $\mathbf{X}$, we need to left-multiply by the pseudo-inverse of $\mathbf{A}$:

$$\hat{\mathbf{X}} = \mathbf{A}^\dagger \mathbf{Y}$$

Let's reconstruct the images in the dataset using the eigenvectors that explain 90% of the variance.

```
In [70]: N_eigenvectors = 63
```

```
# Designing and training dimensionality reduction model
```

```

pipe = Pipeline([('scaler', StandardScaler()),
                  ('pca', PCA(n_components=N_eigenvectors))])
pipe.fit(X_train, t_train)

# Transforming data
ypca = pipe.transform(X_train)

# Reconstructing data
# This includes pseudo_inv(A)*Y and inverse transform for the StandardScaler
X_reconstructed = pipe.inverse_transform(ypca)

# Select 5 random images from training
N = 5
fig = plt.figure(figsize=(15,5))
# idx = np.random.choice(range(X_reconstructed.shape[0]),replace=False,size=
j=1
for i in range(N):
    fig.add_subplot(2,N,j)
    plt.imshow(X_train[idx[i],:].reshape(64,64), cmap='gray')
    plt.axis('off')
    plt.title('Original Image');

    fig.add_subplot(2,N,j+N)
    plt.imshow(X_reconstructed[idx[i],:].reshape(64,64), cmap='gray')
    plt.axis('off')
    plt.title('Reconstructed Image');
    j+=1

```



Introduction to Artificial Neural Networks (ANNs)

A **Multi-Layer Perceptron (MLP)** is a class of feed-forward artificial neural network (ANN). An MLP is composed of one **input layer**, one or more layers of TLUs, called **hidden layers**, and one final layer of TLUs called the **output layer**. Except for the input

nodes, each node is a neuron that uses an activation function (either linear or non-linear).

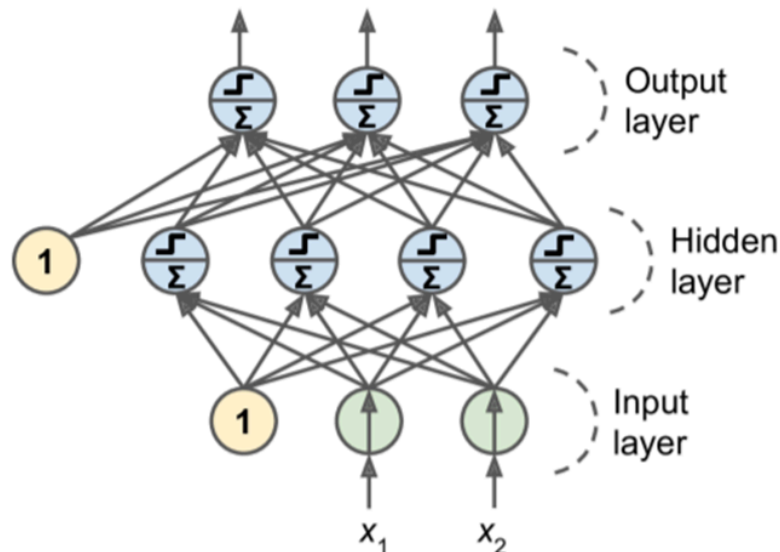
- The layers close to the input layer are usually called the lower layers, and the ones close to the outputs are usually called the upper layers.
- Every layer except the output layer includes a bias neuron and is fully connected to the next layer.

Its multiple layers and non-linear activation distinguish MLP from a linear perceptron. It can distinguish data that is not linearly separable.

When an Artificial Neural Network (ANN) contains a deep stack of hidden layers, it is called a **deep neural network (DNN)**. The field of Deep Learning studies DNNs, and more generally models containing deep stacks of computations.

```
In [2]: Image('figures/MLPs.png',width=400)
#Source: Aurélien Géron, "Hands-on machine learning with scikit-learn, keras"
```

Out [2]:



In MLPs, each neuron's output can be subject to different activation functions. The choice of the activation function is in itself a *hyperparameter*.

- Which activation function would you use if you desired labels are $\{1, 2, 3, 4, \dots, 8, 9\}$?
- What each of the layers do?
 - The first hidden layer draws boundaries
 - The second hidden layer combines the boundaries
 - The third and further layers can generate arbitrarily complex shapes

Types of Activation Functions

In artificial neural networks (ANN), the *activation function* of a neuron defines the output of that neuron given an input or set of inputs.

There are many activation functions, the most common are:

1. Heaviside Step:

$$\phi(x) = \begin{cases} 1, & x > 0 \\ 0, & x \leq 0 \end{cases}$$

2. Linear:

$$\phi(x) = x$$

3. Sigmoid:

$$\phi(x) = \frac{1}{1 + e^{-x}}$$

4. Hyperbolic Tangent (tanh):

$$\phi(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

5. Rectified Linear Unit (ReLU):

$$\phi(x) = \begin{cases} x, & x > 0 \\ 0, & x \leq 0 \end{cases}$$

6. Leaky ReLU:

$$\phi(x) = \begin{cases} x, & x > 0 \\ 0.01x, & x \leq 0 \end{cases}$$

7. Exponential Linear Unit (ELU):

$$\phi(x) = \begin{cases} x, & x > 0 \\ \alpha(e^x - 1), & x \leq 0 \end{cases}$$

where $\alpha = 1.67326$.

8. Scaled Exponential Linear Unit (SELU):

$$\phi(x) = \lambda \begin{cases} x, & x > 0 \\ \alpha(e^x - 1), & x \leq 0 \end{cases}$$

where $\alpha = 1.67326$ and $\lambda = 1.0507$.

9. Softplus:

$$\phi(x) = \ln(1 + e^x)$$

There are many many [other functions](#) but these are the most popular.

```
In [ ]: def activation_functions(x, alpha=1.67326, lam=1.0507):

    linear    = x
    sigmoid   = 1/(1+np.exp(-x))
    tanh      = (np.exp(x)-np.exp(-x))/(np.exp(x)+np.exp(-x))
    softplus  = np.log(1+np.exp(x))

    relu      = x.copy()
    relu[x<=0] = 0

    leaky_relu = x.copy()
    leaky_relu[x<=0]=0.01*x[x<=0]

    elu       = x.copy()
    elu[x<=0] = alpha*(np.exp(x[x<=0])-1)

    selu      = 1.0507*x.copy()
    selu[x<=0] = lam*alpha*(np.exp(x[x<=0])-1)

    functions = np.vstack((linear, sigmoid, tanh, softplus, relu, leaky_relu, elu, selu))
    names     = ['Linear', 'Sigmoid', 'Tanh', 'Softplus', 'ReLU', 'Leaky ReLU', 'ELU', 'SELU']

    return functions, names
```

```
In [ ]: x = np.linspace(-5,5,100)
functions, names = activation_functions(x)

plt.figure(figsize=(15,15))
for i in range(8):
    plt.subplot(4,4,i+1)
    plt.plot(x, functions[:,i], label=names[i])
    plt.legend();
```

Regression and Classification MLPs

Since the Perceptron algorithm was published in 1957, many developments have been made in the design of complex networks.

As mentioned above, an MLP is composed of one input layer, one or more layers of TLUs, called hidden layers, and one final layer of TLUs called the output layer. The layers close to the input layer are usually called the lower layers, and the ones close to the outputs are usually called the upper layers. Every layer except the output layer includes a bias neuron and is fully connected to the next layer.

The connections between neurons are called **weight connections**, or simply weights. And they represent the strength of connection. The larger the coefficient, in the absolute

sense, the more importance that input neuron will have in the neuron it is connected to.

During training, our goal is to find the best values for the weights such the output predicts the desired target value.

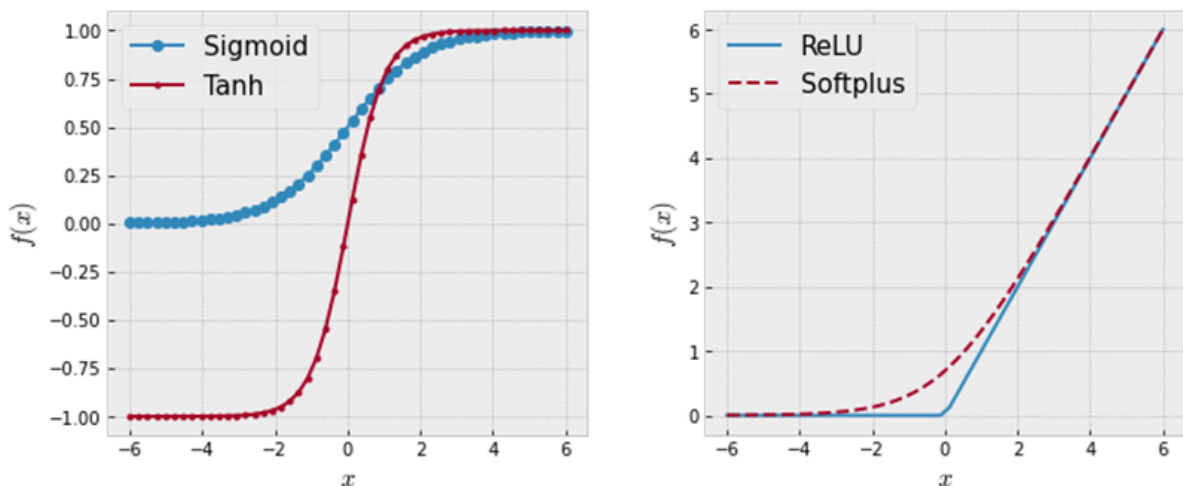
Regression MLPs

MLPs can be used for regression tasks.

- If you want to **predict a single value**, say, for example, the price of a house given many of its features, then you **just need a single output neuron**: its output is the predicted value.
- For **multivariate regression**, say, to predict multiple values at once, you **need one output neuron per output dimension**.
- Consider another example, to locate the center of an object on an image, you need to predict 2D coordinates, so you need two output neurons. If you also want to place a bounding box around the object, then you need two more numbers: the width and the height of the object. So, you end up with 4 output neurons.

```
In [3]: Image('figures/activation_functions.png',width=700)  
#Source: Aurélien Géron, "Hands-on machine learning with scikit-learn, keras"
```

Out [3]:



In general, when building an MLP for regression, you do not want to use any **activation function** for the output neurons, so they are free to output any range of values.

However, if you want to guarantee that the output will always be positive, then you can use the **ReLU activation function**, or the **softplus activation function** in the output layer.

Finally, if you want to guarantee that the predictions will fall within a given range of values, then you can use the **sigmoid** function or the **hyperbolic tangent** and scale the

labels to the appropriate range: 0 to 1 for the logistic function, or -1 to 1 for the hyperbolic tangent.

The loss function to use during training is typically the mean squared error, but if you have a lot of outliers in the training set, you may prefer to use the mean absolute error instead. Alternatively, you can use the Huber loss, which is a combination of both.

Hyperparameter for Regression MLPs

The hyperparameters represent the set of controllable factors during the design of the MLP architecture. Here is an example of the hyperparameter choices for training an MLP for a regression task.

Hyperparameter	Typical Value
# input neurons	One per input feature (e.g., $28 \times 28 = 784$ for MNIST)
# hidden layer	Depends on the problem. Typically, 1 to 5.
# neurons per hidden layer	Depends on the problem. Typically, 10 to 100.
# output neurons	1 per prediction dimension
Hidden activation function	ReLU
Output activation function	None or ReLU/Softplus (if positive outputs) or Logistic/Tanh (if bounded outputs)
Loss function	MSE or MAE/Huber (if outliers)

Classification MLPs

MLPs, and artificial neural networks in general, can perform both regression and classification tasks.

For a **binary classification problem**, you just need a **single output neuron using the sigmoid activation function**: the output will be a number between 0 and 1, which you can interpret as the estimated probability of the positive class. Obviously, the estimated probability of the negative class is equal to one minus that number.

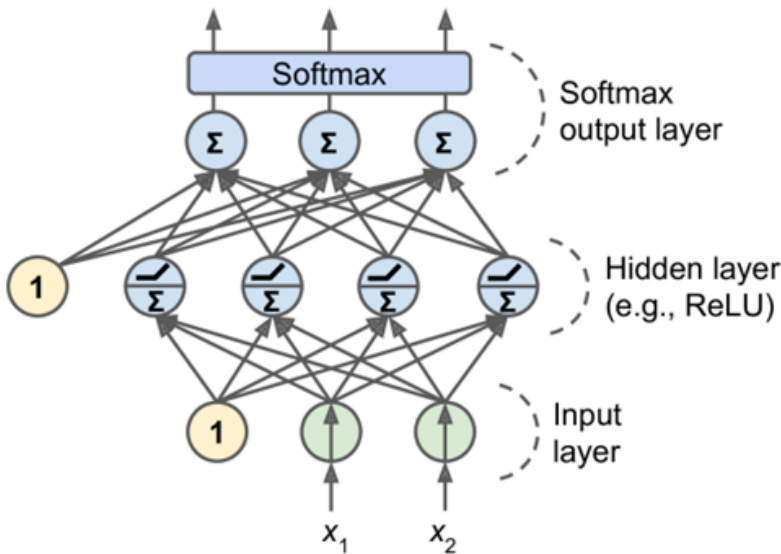
MLPs can also easily handle **multilabel binary classification tasks**. For example, you could have an email classification system that predicts whether each incoming email is non-spam or spam, and simultaneously predicts whether it is an urgent or non-urgent email. In this case, you would need two output neurons, both using the sigmoid activation function: the first would output the probability that the email is spam and the second would output the probability that it is urgent. More generally, you would dedicate one output neuron for each positive class. Note that the output probabilities do not

necessarily add up to one. This lets the model output any combination of labels: you can have non-urgent ham, urgent ham, non-urgent spam, and perhaps even urgent spam (although that would probably be an error).

If each instance can belong only to a single class, out of 3 or more possible classes (for example, classes 0 through 9 for digit image classification), then you need to have one output neuron per class, and you should use the softmax activation function for the whole output layer. The softmax function will ensure that all the estimated probabilities are between 0 and 1 and that they add up to one (which is required if the classes are exclusive). This is called multiclass classification.

```
In [4]: Image('figures/classification_MLPs.png',width=400)
#Source: Aurélien Géron, "Hands-on machine learning with scikit-learn, keras
```

Out [4]:



Hyperparameter	Binary Classification	Multilabel binary classification	Multiclass classification
Input and hidden layers	Same as regression	Same as regression	Same as regression
# output neurons	1	1 per label	1 per class
Output layer activation	Sigmoid	Sigmoid	Softmax
Loss function	Cross-Entropy	Cross-Entropy	Cross-Entropy

Universal Approximation Theorem

"The *Universal Approximation Theorem* states that a feed-forward network with a single hidden layer containing a finite number of neurons can approximate continuous functions on compact subsets of $\mathbb{R}^N$, under mild assumptions on the activation function.

The theorem thus states that simple neural networks can represent a wide variety of interesting functions when given appropriate parameters; however, it does not touch upon the algorithmic learnability of those parameters."

Let $\phi(\cdot)$ be a non-constant, bounded and monotonic-increasing continuous function. Let I_{m_0} denote the m_0 -dimensional unit hypercube $[0, 1]^{m_0}$. The space of continuous functions on I_{m_0} is denoted by $C(I_{m_0})$. Then, given any function $f \in C(I_{m_0})$ and $\epsilon > 0$, there exists an integer m_1 and sets of real constants α_i, β_i , and w_{ij} , where $i = 1, \dots, m_1$ and $j = 1, \dots, m_0$ such that we may define

$$F(x_1, \dots, x_{m_0}) = \sum_{i=1}^{m_1} \alpha_i \phi \left(\sum_{j=1}^{m_0} w_{ij} x_j + \beta_i \right)$$

as an approximation realization of the function $f(\cdot)$, that is,

$$|F(x_1, \dots, x_{m_0}) - f(x_1, \dots, x_{m_0})| < \epsilon$$

for all $x_1, x_2, \dots, x_{m_0}$ that lie in the input space.

Essentially, the Universal Approximation Theorem states that a single hidden layer is sufficient for a multilayer perceptron to compute a uniform ϵ approximation to a given training set - provided you have the *right* number of neurons and the *right* activation function.

- However, this does not say that a single hidden layer is optimal with regards to learning time, generalization, etc.)
 - In other words, a **feed-forward MLP with one hidden layer can approximate arbitrarily closely any continuous function.** (Wow!)
-